

# 第五章 模型架构

在前述章节中已经对预训练数据的准备流程（第 4 章）进行了介绍。本章主要讨论大语言模型的模型架构选择，主要围绕 Transformer 模型（第 5.1 节）、详细配置（第 5.2 节）、主流架构（第 5.3 节）、长上下文模型（第 5.4 节）以及创新型模型（第 5.5 节）等五个主要方面展开讨论。表 5.1 列举了一些典型的大语言模型的详细配置。

**表 5.1** 大语言模型架构配置表（L 表示层数，N 表示注意力头数，H 表示隐藏状态的大小，表格来源：[10]）

| 模型              | 类别  | 大小   | 归一化       | 位置编码     | 激活函数   | L   | N   | H     |
|-----------------|-----|------|-----------|----------|--------|-----|-----|-------|
| GPT-3           | 因果  | 175B | Pre Layer | Learned  | GELU   | 96  | 96  | 12288 |
| PanGU- $\alpha$ | 因果  | 207B | Pre Layer | Learned  | GELU   | 64  | 128 | 16384 |
| OPT             | 因果  | 175B | Pre Layer | Learned  | ReLU   | 96  | 96  | 12288 |
| PaLM            | 因果  | 540B | Pre Layer | RoPE     | SwiGLU | 118 | 48  | 18432 |
| BLOOM           | 因果  | 176B | Pre Layer | ALiBi    | GELU   | 70  | 112 | 14336 |
| MT-NLG          | 因果  | 530B | -         | -        | -      | 105 | 128 | 20480 |
| Gopher          | 因果  | 280B | Pre RMS   | Relative | -      | 80  | 128 | 16384 |
| Chinchilla      | 因果  | 70B  | Pre RMS   | Relative | -      | 80  | 64  | 8192  |
| Galactica       | 因果  | 120B | Pre Layer | Learned  | GELU   | 96  | 80  | 10240 |
| LaMDA           | 因果  | 137B | -         | Relative | GeGLU  | 64  | 128 | 8192  |
| Jurassic-1      | 因果  | 178B | Pre Layer | Learned  | GELU   | 76  | 96  | 13824 |
| LLaMA-2         | 因果  | 70B  | Pre RMS   | RoPE     | SwiGLU | 80  | 64  | 8192  |
| Pythia          | 因果  | 12B  | Pre Layer | RoPE     | GELU   | 36  | 40  | 5120  |
| Baichuan-2      | 因果  | 13B  | Pre RMS   | ALiBi    | SwiGLU | 40  | 40  | 5120  |
| Qwen-1.5        | 因果  | 72B  | Pre RMS   | RoPE     | SwiGLU | 80  | 64  | 8192  |
| InternLM-2      | 因果  | 20B  | Pre RMS   | RoPE     | SwiGLU | 48  | 48  | 6144  |
| Falcon          | 因果  | 180B | Pre Layer | RoPE     | GELU   | 80  | 232 | 14848 |
| MPT             | 因果  | 30B  | Pre Layer | ALiBi    | GELU   | 48  | 64  | 7168  |
| Mistral         | 因果  | 7B   | Pre RMS   | RoPE     | SwiGLU | 32  | 32  | 4096  |
| Gemma           | 因果  | 7B   | Pre RMS   | RoPE     | GELU   | 28  | 16  | 3072  |
| DeepSeek        | 因果  | 67B  | Pre RMS   | RoPE     | SwiGLU | 95  | 64  | 8192  |
| Yi              | 因果  | 34B  | Pre RMS   | RoPE     | SwiGLU | 60  | 56  | 7168  |
| YuLan           | 因果  | 12B  | Pre RMS   | RoPE     | SwiGLU | 40  | 38  | 4864  |
| GLM-130B        | 前缀  | 130B | Post Deep | RoPE     | GeGLU  | 70  | 96  | 12288 |
| T5              | 编-解 | 11B  | Pre RMS   | Relative | ReLU   | 24  | 128 | 1024  |

## 5.1 Transformer 模型

当前主流的大语言模型都基于 Transformer 模型进行设计的。Transformer 是由多层的多头自注意力 (Multi-head Self-attention) 模块堆叠而成的神经网络模型。原始的 Transformer 模型由编码器和解码器两个部分构成，而这两个部分实际上可以独立使用，例如基于编码器架构的 BERT 模型 [13] 和解码器架构的 GPT 模型 [14]。与 BERT 等早期的预训练语言模型相比，大语言模型的特点是使用了更长的向量维度、更深的层数，进而包含了更大规模的模型参数，并主要使用解码器架构，对于 Transformer 本身的结构与配置改变并不大。本部分内容将首先介绍 Transformer 模型的基本组成，包括基础的输入、多头自注意力模块和前置网络层；接着分别介绍 Transformer 模型中的编码器和解码器模块。

### 5.1.1 输入编码

在 Transformer 模型中，输入的词元序列 ( $\mathbf{u} = [u_1, u_2, \dots, u_T]$ ) 首先经过一个输入嵌入模块 (Input Embedding Module) 转化成词向量序列。具体来说，为了捕获词汇本身的语义信息，每个词元在输入嵌入模块中被映射成为一个可学习的、具有固定维度的词向量  $\mathbf{v}_t \in \mathbb{R}^H$ 。由于 Transformer 的编码器结构本身无法识别序列中元素的顺序，位置编码 (Position Embedding, PE) 被引入来表示序列中的位置信息。给定一个词元  $u_t$ ，位置编码根据其在输入中的绝对位置分配一个固定长度的嵌入向量  $\mathbf{p}_t \in \mathbb{R}^H$ 。然后，每个词元对应的词向量和位置向量将直接相加，生成了最终的输入嵌入序列  $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_T]$ ，并且被传入到后续层中：

$$\mathbf{x}_t = \mathbf{v}_t + \mathbf{p}_t. \quad (5.1)$$

通过这种建模方法的表示，Transformer 模型可以利用位置编码  $\mathbf{p}_t$  建模不同词元的位置信息。由于不同词元的位置编码仅由其位置唯一决定，因此这种位置建模方式被称为绝对位置编码。尽管绝对位置编码能够一定程度上建模位置信息，然而它只能局限于建模训练样本中出现的位置，无法建模训练数据中未出现过的位

### 5.1.2 多头自注意力机制

多头自注意力是 Transformer 模型的核心创新技术。相比于循环神经网络 (Recurrent Neural Network, RNN) 和卷积神经网络 (Convolutional Neural Network, CNN) 等传统神经网络, 多头自注意力机制能够直接建模任意距离的词元之间的交互关系。作为对比, 循环神经网络迭代地利用前一个时刻的状态更新当前时刻的状态, 因此在处理较长序列的时候, 常常会出现梯度爆炸或者梯度消失的问题。而在卷积神经网络中, 只有位于同一个卷积核的窗口中的词元可以直接进行交互, 通过堆叠层数来实现远距离词元间信息的交换。

多头自注意力机制通常由多个自注意力模块组成。在每个自注意力模块中, 对于输入的词元序列, 将其映射为相应的查询 (Query,  $Q$ )、键 (Key,  $K$ ) 和值 (Value,  $V$ ) 三个矩阵。然后, 对于每个查询, 将和所有没有被掩盖的键之间计算点积。这些点积值进一步除以  $\sqrt{D}$  进行缩放 ( $D$  是键对应的向量维度), 被传入到 softmax 函数中用于权重的计算。进一步, 这些权重将作用于与键相关联的值, 通过加权的形式计算得到最终的输出。在数学上, 上述过程可以表示为:

$$Q = XW^Q, \quad (5.2)$$

$$K = XW^K, \quad (5.3)$$

$$V = XW^V, \quad (5.4)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V. \quad (5.5)$$

与单头注意力相比, 多头注意力机制的主要区别在于它使用了  $H$  组结构相同但映射参数不同的自注意力模块。输入序列首先通过不同的权重矩阵被映射为一组查询、键和值。每组查询、键和值的映射构成一个“头”, 并独立地计算自注意力的输出。最后, 不同头的输出被拼接在一起, 并通过一个权重矩阵  $W^O \in \mathbb{R}^{H \times H}$  进行映射, 产生最终的输出。如下面的公式所示:

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_N)W^O, \quad (5.6)$$

$$\text{head}_n = \text{Attention}(XW_n^Q, XW_n^K, XW_n^V). \quad (5.7)$$

由上述内容可见, 自注意力机制能够直接建模序列中任意两个位置之间的关系, 进而有效捕获长程依赖关系, 具有更强的序列建模能力。另一个主要的优势是, 自注意力的计算过程对于基于硬件的并行优化 (如 GPU、TPU 等) 非常友好, 因此能够支持大规模参数的高效优化。

### 5.1.3 前馈网络层

为了学习复杂的函数关系和特征，Transformer 模型引入了一个前馈网络层 (Feed Forward Netwok, FFN)，对于每个位置的隐藏状态进行非线性变换和特征提取。具体来说，给定输入  $\mathbf{x}$ ，Transformer 中的前馈神经网络由两个线性变换和一个非线性激活函数组成：

$$\text{FFN}(\mathbf{X}) = \sigma(\mathbf{X}\mathbf{W}^U + \mathbf{b}_1)\mathbf{W}^D + \mathbf{b}_2, \quad (5.8)$$

其中  $\mathbf{W}^U \in \mathbb{R}^{H \times H'}$  和  $\mathbf{W}^D \in \mathbb{R}^{H' \times H}$  分别是第一层和第二层的线性变换权重矩阵， $\mathbf{b}_1 \in \mathbb{R}^{H'}$  和  $\mathbf{b}_2 \in \mathbb{R}^H$  是偏置项， $\sigma$  是激活函数（在原始的 Transformer 中，采用 ReLU 作为激活函数）。前馈网络层通过激活函数引入了非线性映射变换，提升了模型的表达能力，从而更好地捕获复杂的交互关系。

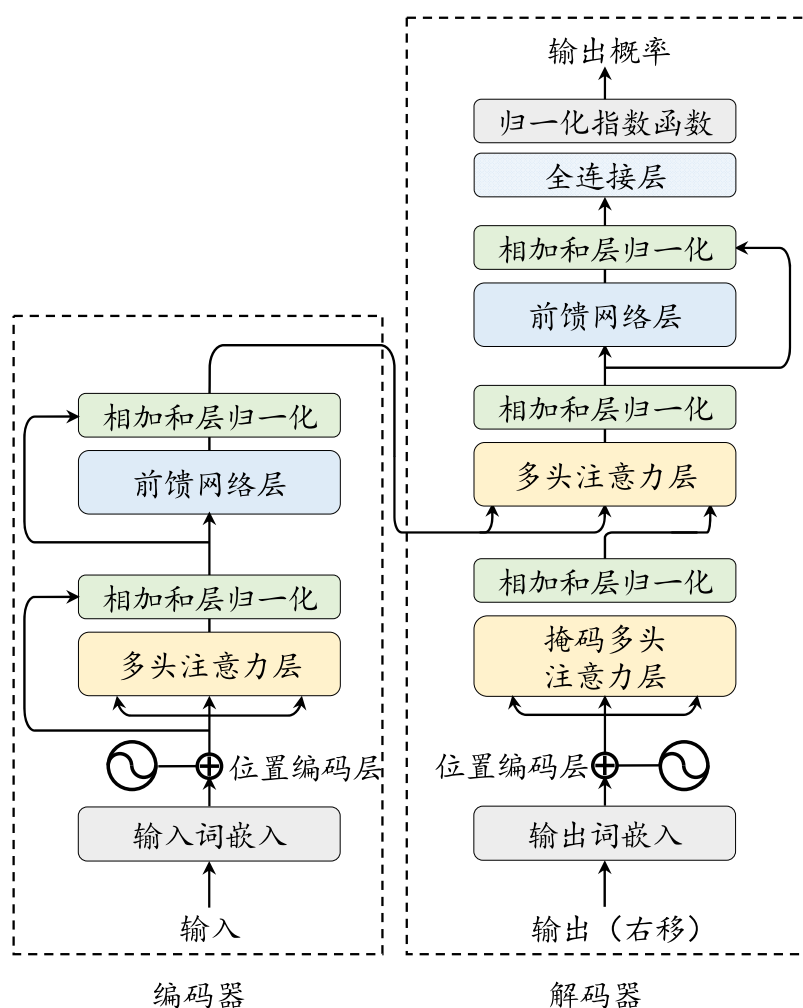


图 5.1 Transformer 架构图



### 5.1.4 编码器

在 Transformer 模型中，编码器（Encoder）（图 5.1 (a)）的作用是将每个输入词元都编码成一个上下文语义相关的表示向量。编码器结构由多个相同的层堆叠而成，其中每一层都包含多头自注意力模块和前馈网络模块。在注意力和前馈网络后，模型使用层归一化和残差连接来加强模型的训练稳定度。其中，残差连接（Residual Connection）将输入与该层的输出相加，实现了信息在不同层的跳跃传递，从而缓解梯度爆炸和消失的问题。而 LayerNorm 则对数据进行重新放缩，提升模型的训练稳定性（详细介绍可见第 5.2.1 节）。编码器接受经过位置编码层的词嵌入序列  $\mathbf{X}$  作为输入，通过多个堆叠的编码器层来建模上下文信息，进而对于整个输入序列进行编码表示。由于输入数据是完全可见的，编码器中的自注意力模块通常采用双向注意力，每个位置的词元表示能够有效融合上下文的语义关系。在编码器-解码器架构中，编码器的输出将作为解码器（Decoder）的输入，进行后续计算。形式化来说，第  $l$  层（ $l \in \{1, \dots, L\}$ ）的编码器的数据处理过程如下所示：

$$\begin{aligned} \mathbf{X}'_l &= \text{LayerNorm}(\text{MHA}(\mathbf{X}_{l-1}) + \mathbf{X}_{l-1}), \\ \mathbf{X}_l &= \text{LayerNorm}(\text{FFN}(\mathbf{X}'_l) + \mathbf{X}'_l), \end{aligned} \quad (5.9)$$

其中， $\mathbf{X}_{l-1}$  和  $\mathbf{X}_l$  分别是该 Transformer 层的输入和输出， $\mathbf{X}'_l$  是该层中输入经过多头注意力模块后的中间表示，LayerNorm 表示层归一化。

### 5.1.5 解码器

Transformer 架构中的解码器（图 5.1 (b)）基于来自编码器编码后的最后一层的输出表示以及已经由模型生成的词元序列，执行后续的序列生成任务。与编码器不同，解码器需要引入掩码自注意力（Masked Self-attention）模块，用来在计算注意力分数的时候掩盖当前位置之后的词，以保证生成目标序列时不依赖于未来的信息。除了建模目标序列的内部关系，解码器还引入了与编码器相关联的多头注意力层，从而关注编码器输出的上下文信息  $\mathbf{X}_L$ 。同编码器类似，在每个模块之后，Transformer 解码器也采用了层归一化和残差连接。在经过解码器之后，模型会通过一个全连接层将输出映射到大小为  $V$  的目标词汇表的概率分布，并基于某种解码策略生成对应的词元。在训练过程中，解码器可以通过一次前向传播，让每个词元的输出用于预测下一个词元。而在解码过程，解码器需要经过一个逐步的生成过程，将自回归地生成完整的目标序列（具体可以参考第 9 章）。解码器的

数据流程如下所示：

$$\begin{aligned} Y'_l &= \text{LayerNorm}(\text{MaskedMHA}(Y_{l-1}) + Y_{l-1}), \\ Y''_l &= \text{LayerNorm}(\text{CrossMHA}(Y'_l, X_L) + Y'_l), \\ Y_l &= \text{LayerNorm}(\text{FFN}(Y''_l) + Y''_l), \end{aligned} \quad (5.10)$$

其中， $Y_{l-1}$  和  $Y_l$  分别是该 Transformer 层的输入和输出， $Y'_l$  和  $Y''_l$  是该层中输入经过掩码多头注意力 MaskedMHA 和交叉多头注意力 CrossMHA 模块后的中间表示，LayerNorm 表示层归一化。然后将最后一层的输入  $Y_L$  映射到词表的维度上：

$$O = \text{softmax}(W^L Y_L), \quad (5.11)$$

其中， $O \in \mathbb{R}^{H \times V}$  是模型最终的输出，代表下一个词在词表上的概率分布； $W^L \in \mathbb{R}^{H \times V}$  是将输入表示映射到词汇表维度的参数矩阵，而  $W^L Y_L$  是概率化前的中间值，通常被称为 logits。

## 5.2 详细配置

自从 Transformer 模型 [12] 公开发布以来，研究人员针对训练稳定性、性能与计算效率提升等方面提出了多种改进方法。本节主要探讨了 Transformer 模型四个核心组件的配置，包括归一化、位置、激活函数和注意力机制，并介绍混合专家结构。最后，我们将通过演示代码对于 LLaMA 的模型实现进行介绍。

### 5.2.1 归一化方法

大语言模型的预训练过程中经常会出现不稳定的问题。为了应对这一问题，深度学习方法通常会采用特定的归一化策略来加强神经网络训练过程的稳定性。原始的 Transformer 模型主要使用了层归一化方法 (Layer Normalization, LN) [158]。随着研究工作的不断深入，基于层归一化的改进技术不断涌现，例如均方根层归一化 (Root Mean Square Layer Normalization, RMSNorm) [159] 和 DeepNorm [160]，这些新技术已经在一些大语言模型中得到应用。下面进行具体介绍。

- *LayerNorm*. 在早期的研究中，批次归一化 (Batch Normalization, BN) [161] 是一种广泛采用的归一化方法。然而，该方法难以处理可变长度的序列数据和小批次数据。因此，相关研究提出了层归一化这一技术 [158]，针对数据进行逐层归一化。具体而言，层归一化会计算每一层中所有激活值的均值  $\mu$  和方差  $\sigma$ ，从而

重新调整激活值的中心和缩放比例:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \cdot \gamma + \boldsymbol{\beta}, \quad (5.12)$$

$$\boldsymbol{\mu} = \frac{1}{H} \sum_{i=1}^H x_i, \quad \boldsymbol{\sigma} = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \boldsymbol{\mu})^2}. \quad (5.13)$$

• *RMSNorm*. 为了提高层归一化的训练速度, RMSNorm [159] 仅利用激活值总和的均方根  $\text{RMS}(\mathbf{x})$  对激活值进行重新缩放。使用 RMSNorm 的 Transformer 模型相比于之前 LayerNorm 训练的模型在训练速度和性能上均具有一定优势。采用 RMSNorm 的代表性模型包括 Gopher [123] 和 Chinchilla [22]。其计算公式如下所示:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma, \quad (5.14)$$

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{H} \sum_{i=1}^H x_i^2}. \quad (5.15)$$

下面给出了 Transformers 代码库中 LLaMA 的 RMSNorm 实现代码:

```
1 class LlamaRMSNorm(nn.Module):
2     def __init__(self, hidden_size, eps=1e-6):
3         super().__init__()
4         self.weight = nn.Parameter(torch.ones(hidden_size))
5         self.variance_epsilon = eps
6
7     def forward(self, hidden_states):
8         input_dtype = hidden_states.dtype
9         hidden_states = hidden_states.to(torch.float32)
10        variance = hidden_states.pow(2).mean(-1, keepdim=True)
11        # 计算隐状态的均方根
12        hidden_states = hidden_states * torch.rsqrt(variance +
13        ↪ self.variance_epsilon)
14        # 将隐状态除以其均方根后重新缩放
15        return self.weight * hidden_states.to(input_dtype)
```

• *DeepNorm*. DeepNorm 由微软的研究人员提出 [160], 旨在稳定深层 Transformer 的训练。具体而言, DeepNorm 在 LayerNorm 的基础上, 在残差连接中对之前的激活值  $\mathbf{x}$  按照一定比例  $\alpha$  进行放缩。通过这一简单的操作, Transformer 的层数可以被成功地扩展至 1,000 层 [160], 进而有效提升了模型性能与训练稳定性。其计算公式如下:

$$\text{DeepNorm}(\mathbf{x}) = \text{LayerNorm}(\alpha \cdot \mathbf{x} + \text{Sublayer}(\mathbf{x})), \quad (5.16)$$

其中, Sublayer 表示 Transformer 层中的前馈神经网络或自注意力模块。GLM-130B [162]

采用了 DeepNorm 作为归一化技术。

### 5.2.2 归一化模块位置

为了加强大语言模型训练过程的稳定性，除了归一化方法外，归一化模块的位置也具有重要的影响。如图 5.2(a) 所示，归一化模块的位置通常有三种选择，分别是层后归一化（Post-Layer Normalization, Post-Norm）、层前归一化（Pre-Layer Normalization, Pre-Norm）和夹心归一化（Sandwich-Layer Normalization, Sandwich-Norm）。

- *Post-Norm.* Post-Norm 是在原始 Transformer 模型中所使用的一种归一化技术。其中，归一化模块被放置于残差计算之后。其计算公式如下：

$$\text{Post-Norm}(\mathbf{x}) = \text{Norm}(\mathbf{x} + \text{Sublayer}(\mathbf{x})), \quad (5.17)$$

其中，Norm 表示任意一种归一化方法。在原理上，后向归一化具有很多优势。首先，有助于加快神经网络的训练收敛速度，使模型可以更有效地传播梯度，从而减少训练时间。其次，后向归一化可以降低神经网络对于超参数（如学习率、初始化参数等）的敏感性，使得网络更容易调优，并减少了超参数调整的难度。然而，由于在输出层附近存在梯度较大的问题，采用 Post-Norm 的 Transformer 模型在训练过程中通常会出现不稳定的现象 [163]。因此，现有的大语言模型中，Post-Norm 很少被单独使用，通常是与其他策略相结合应用。例如，GLM-130B 将 Post-Norm 与 DeepNorm 结合使用。

- *Pre-Norm.* 与 Post-Norm 不同，Pre-Norm [164] 将归一化模块应用在每个子层之前。其计算公式如下：

$$\text{Pre-Norm}(\mathbf{x}) = \mathbf{x} + \text{Sublayer}(\text{Norm}(\mathbf{x})), \quad (5.18)$$

此处的 Norm 泛指任意一种归一化方法。此外，Pre-Norm 在最后一个 Transformer 层后还额外添加了一个 LayerNorm。相较于 Post-Norm，Pre-Norm 直接把每个子层加在了归一化模块之后，仅仅对输入的进行进行了归一化，从而可以防止模型的梯度爆炸或者梯度消失现象。虽然使用了 Pre-Norm 的 Transformer 模型在训练过程中更加稳定，但是性能却逊色于采用了 Post-Norm 的模型。尽管对于性能有一定的影响，但由于其能够有效维持训练的稳定性，很多主流的大语言模型仍然采用 Pre-Norm。

- *Sandwich-Norm.* 在 Pre-Norm 的基础上，Sandwich-Norm [165] 在残差连接之

前增加了额外的 LayerNorm，旨在避免 Transformer 层的输出出现数值爆炸的情况。具体的实现方式如下所示：

$$\text{Sandwich-Norm}(\mathbf{x}) = \mathbf{x} + \text{Norm}(\text{Sublayer}(\text{Norm}(\mathbf{x}))). \quad (5.19)$$

本质上，Sandwich-Norm 可以看作是 Pre-Norm 和 Post-Norm 两种方法的组合，理论上具有更加灵活的表达能力。但是研究人员发现，Sandwich-Norm 有时仍然无法保证大语言模型的稳定训练，甚至会引发训练崩溃的问题 [162]。

### 5.2.3 激活函数

前馈网络中激活函数的选择对于大语言模型的表现至关重要。通常来说，激活函数主要是为神经网络中引入非线性变化，从而提升神经网络的模型能力。在原始的 Transformer 中采用了 ReLU (Rectified Linear Unit) 激活函数。该激活函数计算较为简单，仅仅是将对输入中每个神经元和“零值”进行比较，并将小于零的神经元的值设置为 0。然而，ReLU 可能会产生神经元失效的问题，被置为 0 的神经元将学习不到有用的信息。ReLU 函数的具体形式如下所示：

$$\text{ReLU}(x) = \max(x, 0). \quad (5.20)$$

针对 ReLU 存在的不足，研究人员进一步探索了 ReLU 函数的变种，以实现更好的性能。Swish 激活函数将神经元和该神经元的 sigmoid 激活的乘积作为新的激活函数。而 GELU (Gaussian Error Linear Unit) [166] 则利用标准高斯累积分布函数作为激活函数，被很多的 Transformer 模型所采用。相比于原始的 ReLU 函数，这些新的激活函数通常能够带来更好的性能并且收敛性更好，但是计算过程更为复杂。Swish 和 GELU 与 ReLU 的对比如图 5.2(b) 所示。Swish 和 GELU 的数学表示如下：

$$\text{Swish}(x) = x \cdot \text{sigmoid}(x), \quad (5.21)$$

$$\text{GELU}(x) = 0.5x \cdot [1 + \text{erf}(x/\sqrt{2})], \quad \text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_1^x e^{-t^2} dt. \quad (5.22)$$

近来，大语言模型（例如 PaLM 和 LaMDA）也经常采用 GLU (Gated Linear Unit) 激活函数以及它的变种 [167]，特别是 SwiGLU 和 GeGLU。不同于其他激活函数，GLU 激活函数引入了两个不同的线性层。其中一个线性层的输出将被输入到一个激活函数（例如，GeGLU 采用 GELU 激活函数）中，其结果将和另一个线性层的输出进行逐元素相乘作为最终的输出。相比于其他的激活函数，使用 GLU

激活函数变体通常能够带来更佳的性能表现 [168]。SwiGLU 和 GeGLU 激活函数的计算公式如下所示：

$$\text{SwiGLU}(\mathbf{x}) = \text{Swish}(\mathbf{W}^G \mathbf{x}) \odot (\mathbf{W}^U \mathbf{x}), \quad (5.23)$$

$$\text{GeGLU}(\mathbf{x}) = \text{GELU}(\mathbf{W}^G \mathbf{x}) \odot (\mathbf{W}^U \mathbf{x}). \quad (5.24)$$

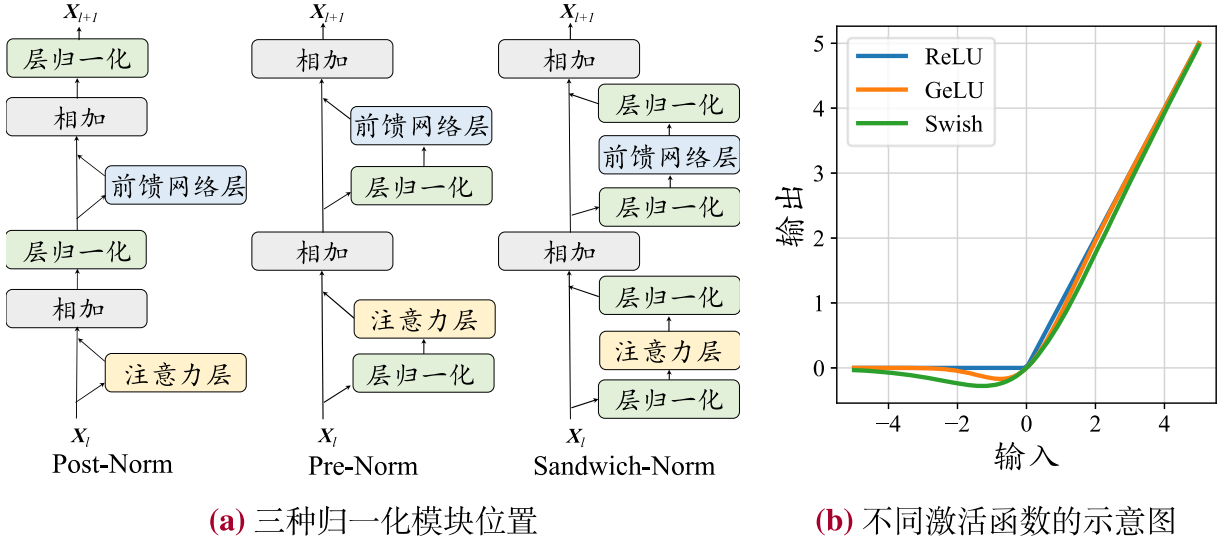


图 5.2 归一化和激活函数的示意图

### 5.2.4 位置编码

由于 Transformer 模型中自注意力模块具有置换不变性，因此仅使用注意力机制无法捕捉序列中的顺序关系，从而退化为“词袋模型”。为了解决这一问题，需要引入位置编码（Position Embedding, PE）对于序列信息进行精确建模，从而将绝对或相对位置信息整合到模型中。

- **绝对位置编码**. 在原始的 Transformer 模型中，为了处理序列数据的顺序信息，采用了绝对位置编码方法。在编码器和解码器的输入端，根据输入的词元在序列中的绝对位置生成唯一的位置嵌入，并与词元的嵌入表示进行相加来注入位置信息。绝对位置编码的公式如下所示：

$$\mathbf{x}_t = \mathbf{v}_t + \mathbf{p}_t, \quad (5.25)$$

其中， $\mathbf{p}_t$  表示位置  $t$  的位置嵌入， $\mathbf{v}_t$  是该位置词元对应的词向量。原始的 Transformer 采用了正余弦位置编码。该位置编码在不同维度上预先定义了特定的正弦或余弦函数，通过将词元的绝对位置作为输入代入这些函数，从而为这些维度赋予相应的值。对于维度大小为  $H$  的位置嵌入，其第  $i \in \{1, \dots, H\}$  维的值按照如下



方法进行设置：

$$p_{t,i} = \begin{cases} \sin(t/10000^{(i-2)/H}) & i \bmod 2 = 0, \\ \cos(t/10000^{(i-1)/H}) & i \bmod 2 = 1. \end{cases} \quad (5.26)$$

此外，绝对位置编码还可以采用可学习的嵌入表示，并被很多早期的预训练语言模型（如 BERT）广泛采用。

- **相对位置编码**. 与绝对位置编码不同，相对位置编码是根据键和查询之间的偏移量计算得来的。计算得到的相对位置编码通常应用于注意力矩阵的计算中，而不是直接与词元本身的位置编码进行相加。其中，Transformer-XL [169] 提出了一种相对位置编码方法，在计算键和查询之间的注意力分数时引入了相对位置信息。对于使用绝对位置编码的模型，其注意力值可以进行进一步的分解：

$$\begin{aligned} A_{ij} &= \mathbf{x}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{x}_j^\top \\ &= (\mathbf{v}_i + \mathbf{p}_i) \mathbf{W}^Q \mathbf{W}^{K\top} (\mathbf{v}_j + \mathbf{p}_j)^\top \\ &= \mathbf{v}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{v}_j^\top + \mathbf{v}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{p}_j^\top + \mathbf{p}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{v}_j^\top + \mathbf{p}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{p}_j^\top. \end{aligned} \quad (5.27)$$

而 Transformer-XL 对上述注意力值进行了改写，使用相对位置信息代替绝对位置信息。其公式表示如下所示（这里使用不同颜色与原始项进行对应）：

$$A_{ij} = \mathbf{x}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{x}_j^\top + \mathbf{x}_i \mathbf{W}^Q \mathbf{W}^{R\top} \mathbf{r}_{i-j}^\top + \mathbf{u} \mathbf{W}^{K\top} \mathbf{x}_j^\top + \mathbf{v} \mathbf{W}^{R\top} \mathbf{r}_{i-j}^\top, \quad (5.28)$$

其中， $\mathbf{x}_i$  是每个词元对应的词向量（对应没有显式加入位置编码的词向量  $\mathbf{v}_i$ ），而  $\mathbf{r}_{i-j}$  表示相对位置编码， $\mathbf{u}$  和  $\mathbf{v}$  是两个可学习的表示全局信息的参数。相比于绝对位置编码，注意力值的第二项中和第四项键对应的绝对位置编码  $\mathbf{W}^{K\top} \mathbf{p}_j$  被替换为相对位置编码  $\mathbf{r}_j$ ，以引入相对位置信息；而第三和第四项中则使用全局参数  $\mathbf{u}$  和  $\mathbf{v}$  替换查询对应的绝对位置编码  $\mathbf{p}_i \mathbf{W}^Q$ ，用于衡量键的语义信息和相对位置信息本身的重要程度。作为另一种方法，T5 [77] 提出了一种较为简化的相对位置编码。具体来说，它在注意力分数中引入了可学习的标量，这些标量是基于查询和键的位置之间的距离计算的。与绝对位置编码相比，应用了相对位置编码的 Transformer 模型常常可以对比训练序列更长的序列进行建模，即具备一定的 extrapolation 能力 [170]。T5 相对位置编码的计算可以表达为：

$$A_{ij} = \mathbf{x}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{x}_j^\top + r_{i-j}, \quad (5.29)$$

其中  $r_{i-j}$  表示基于查询和键之间偏移的可学习标量。

- **旋转位置编码 (Rotary Position Embedding, RoPE)**. RoPE 巧妙地使用了基于

绝对位置信息的旋转矩阵来表示注意力中的相对位置信息。RoPE 根据位置信息为序列中每个词元所对应的设置了独有的旋转矩阵，并和对应的查询和键进行相乘进行融合。形式化，位置索引为  $t$  对应的旋转矩阵定义如下所示：

$$\mathbf{R}_{\theta,t} = \begin{bmatrix} \cos t\theta_1 & -\sin t\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin t\theta_1 & \cos t\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos t\theta_2 & -\sin t\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin t\theta_2 & \cos t\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos t\theta_{H/2} & -\sin t\theta_{H/2} \\ 0 & 0 & 0 & 0 & \dots & \sin t\theta_{H/2} & \cos t\theta_{H/2} \end{bmatrix}. \quad (5.30)$$

利用旋转矩阵中三角函数的特性，位置索引为  $i$  的旋转矩阵和位置索引为  $j$  的旋转矩阵的转置的乘积等同于位置索引为它们相对距离  $i-j$  的旋转矩阵，即  $\mathbf{R}_{\theta,i}\mathbf{R}_{\theta,j}^\top = \mathbf{R}_{\theta,i-j}$ 。通过这种方式，键和查询之间的注意力分数能够有效融入相对位置信息。注意力矩阵的公式可以进一步变为如下形式：

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta,i}, \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K \mathbf{R}_{\theta,j}, \quad (5.31)$$

$$A_{ij} = (\mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta,i})(\mathbf{x}_j \mathbf{W}^K \mathbf{R}_{\theta,j})^\top = \mathbf{x}_i \mathbf{W}_q \mathbf{R}_{\theta,i-j} \mathbf{W}_k^\top \mathbf{x}_j^\top.$$

根据旋转矩阵的定义，RoPE 在处理查询和键向量的时候，将每对连续出现的两个元素视为一个子空间。因此，对于一个长度为  $H$  的向量来说，将会形成  $H/2$  个这样的子空间。在这些子空间中，每一个子空间  $i \in \{1, \dots, H/2\}$  所对应的两个元素都会根据一个特定的旋转角度  $t \cdot \theta_i$  进行旋转，其中  $t$  代表位置索引，而  $\theta_i$  表示该子空间中的基。与正弦位置嵌入类似 [12]，RoPE 将基  $\theta_i$  定义为底数  $b$ （默认值是 10000）的指数：

$$\Theta = \{\theta_i = b^{-2(i-1)/H} | i \in \{1, 2, \dots, H/2\}\}. \quad (5.32)$$

进一步，每个子空间定义了波长  $\lambda_i$ ，即在该子空间上完成一个完整周期 ( $2\pi$ ) 旋转所需的距离：

$$\lambda_i = 2\pi b^{2(i-1)/H} = 2\pi/\theta_i. \quad (5.33)$$

由于 RoPE 具有良好的性能以及长期衰减的特性，已经主流的大语言模型广泛采用，例如 PaLM [33] 和 LLaMA [34]。这里给出了 Transformers 代码库中 LLaMA 的 RoPE 实现代码：

```

1 def rotate_half(x):
2     x1 = x[..., : x.shape[-1] // 2]
3     x2 = x[..., x.shape[-1] // 2 :]
4     # 将向量每两个元素视为一个子空间
5     return torch.cat((-x2, x1), dim=-1)
6
7 def apply_rotary_pos_emb(q, k, cos, sin, position_ids):
8     cos = cos[position_ids].unsqueeze(1)
9     sin = sin[position_ids].unsqueeze(1)
10    # 获得各个子空间旋转的正余弦值
11    q_embed = (q * cos) + (rotate_half(q) * sin)
12    k_embed = (k * cos) + (rotate_half(k) * sin)
13    # 将每个子空间按照特定角度进行旋转
14    return q_embed, k_embed

```

• **ALiBi 位置编码**: ALiBi [170] 是一种特殊的相对位置编码，主要用于增强 Transformer 模型的外推能力。具体来说，ALiBi 通过在键和查询之间的距离上施加相对距离相关的惩罚来调整注意力分数。其计算公式如下：

$$A_{ij} = \mathbf{x}_i \mathbf{W}^Q \mathbf{W}^{K\top} \mathbf{x}_j^\top - m(i - j), \quad (5.34)$$

其中， $i - j$  是查询和键之间的位置偏移量， $m$  是每个注意力头独有的惩罚系数。与 T5 [77] 等模型中的相对位置编码不同，ALiBi 中的惩罚分数是预先设定的，不需要引入任何可训练的参数。此外，ALiBi 展现出了优秀的外推性能，能够对于超过上下文窗口更远距离的词元进行有效建模。下面给出 Transformers 库中 BLOOM 中的 ALiBi 代码实现：

```

1 def build_alibi_tensor(attention_mask: torch.Tensor, num_heads: int, dtype:
↪ torch.dtype) -> torch.Tensor:
2     batch_size, seq_length = attention_mask.shape
3     closest_power_of_2 = 2 ** math.floor(math.log2(num_heads))
4     base = torch.tensor(
5         2 ** (-(2 ** -(math.log2(closest_power_of_2) - 3))),
6         ↪ device=attention_mask.device, dtype=torch.float32
7     )
8     powers = torch.arange(1, 1 + closest_power_of_2,
9         ↪ device=attention_mask.device, dtype=torch.int32)
10    slopes = torch.pow(base, powers)
11    # 计算各个头的惩罚系数
12
13    if closest_power_of_2 != num_heads:
14        # 如果头数不是 2 的幂次方，修改惩罚系数
15        extra_base = torch.tensor(
16            2 ** (-(2 ** -(math.log2(2 * closest_power_of_2) - 3))),
17            ↪ device=attention_mask.device, dtype=torch.float32
18        )
19        num_remaining_heads = min(closest_power_of_2, num_heads -
20            ↪ closest_power_of_2)
21        extra_powers = torch.arange(1, 1 + 2 * num_remaining_heads, 2,
22            ↪ device=attention_mask.device, dtype=torch.int32)

```

```

18     slopes = torch.cat([slopes, torch.pow(extra_base, extra_powers)],
    ↪     dim=0)
19
20     arange_tensor = ((attention_mask.cumsum(dim=-1) - 1) *
    ↪     attention_mask)[:, None, :]
21     # 计算相对距离
22     alibi = slopes[..., None] * arange_tensor
23     # 计算 ALiBi 施加的注意力偏置
24     return alibi.reshape(batch_size * num_heads, 1, seq_length).to(dtype)

```

### 5.2.5 注意力机制

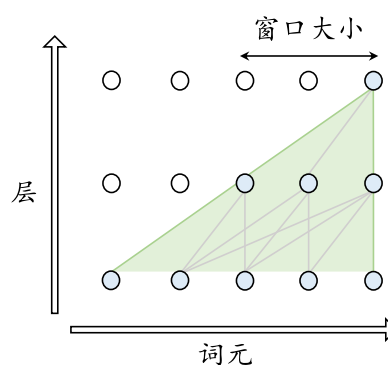
注意力机制是 Transformer 架构中的核心技术，它能够针对序列中的词元对构建交互关系，聚合来自于不同位置的语义信息。下面介绍四种常见的注意力机制的设计方法。

- 完整自注意力机制. 在原始的 Transformer 模型中，注意力机制通过成对的方式进行序列数据的语义建模，充分考虑了序列中所有词元之间的相互关系。其中，每个词元在注意力计算中都需要对于其前序的所有词元的键值对予以访问，因此对于序列长度为  $T$  的序列需要  $O(T^2)$  的计算复杂度。此外，Transformer 还引入了多头注意力机制，将查询、键和值在不同的语义空间进行线性投影，然后将每个头的输出进行聚合形成最终的输出。对于完整注意力的详细介绍，可以参考第 5.1.2 节。

- 稀疏注意力机制. 尽管完整自注意力机制具有较强的建模能力，但是它需要平方级的计算复杂性。在处理长序列时较为显著，带来了较大的计算和存储开销。为了降低注意力机制的计算复杂度，研究人员提出了多种高效的注意力变种。其中，滑动窗口注意力机制（Sliding Window Attention, SWA）是大语言模型中使用最多的一种稀疏注意力机制。不同于完整的注意力机制，滑动窗口注意力根据词元位置，仅仅将位置索引上距离该词元一定范围内的词元考虑到注意力的计算中。具体来说，滑动窗口注意力设置了一个大小为  $w$  的窗口，对每个词元  $u_t$ ，只对窗口内的词元  $[u_{t-w+1}, \dots, u_t]$  进行注意力计算，从而将复杂度降低到  $O(wT)$ 。进一步，通过信息的逐层传递，模型实现了随着层数线性增长的感受野，从而获取远处词元的信息。关于滑动窗口注意力详细机制如图 5.3 展示。

- 多查询/分组查询注意力. 为了提升注意力机制的效率，多查询注意力（Multi-Query Attention, MQA）提出针对不同的头共享相同的键和值变换矩阵 [171]。这种方法减少了访存量，提高了计算强度，从而实现了更快的解码速度（具体可以参

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 中 | 国 | 人 | 民 | 大 | 学 |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 1 | 1 |



(a) 滑动窗口注意力的掩码矩阵

(b) 滑动窗口注意力信息的逐层传递

图 5.3 滑动窗口注意力示意图

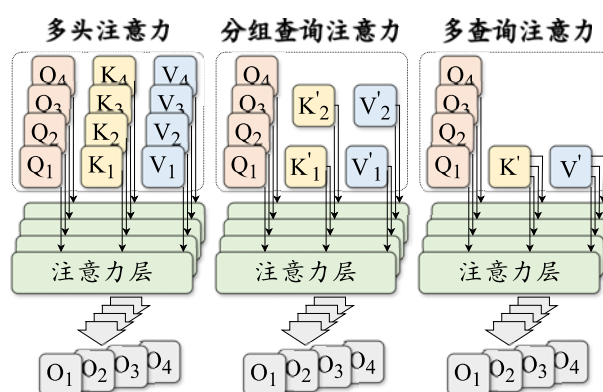


图 5.4 多头注意力、分组查询注意力和多查询注意力示意图

考第 9.2.1 节), 并且对于模型性能产生的影响也比较小。一些代表性的大语言模型, 如 PaLM [33] 和 StarCoder [96], 已经使用了多查询注意力机制。为了结合多查询注意力机制的效率与多头注意力机制的性能, 研究人员进一步提出了分组查询注意力机制 (Grouped-Query Attention, GQA) [172]。GQA 将全部的头划分为若干组, 并且针对同一组内的头共享相同的变换矩阵。这种注意力机制有效地平衡了效率和性能, 被 LLaMA-2 模型所使用。图 5.4 展示了上述两种注意力查询机制。

- 硬件优化的注意力机制. 除了在算法层面上提升注意力机制的计算效率, 还可以进一步利用硬件设施来优化注意力模块的速度和内存消耗。其中, 两个具有代表性的工作是 FlashAttention [173] 与 PagedAttention [174]。相比于传统的注意力实现方式, FlashAttention 通过矩阵分块计算以及减少内存读写次数的方式, 提高注意力分数的计算效率; PagedAttention 则针对增量解码阶段, 对于 KV 缓存进行分块存储, 并优化了计算方式, 增大了并行计算度, 从而提高了计算效率。对于这些技术的细节将在第 9.2.2 节进行介绍。

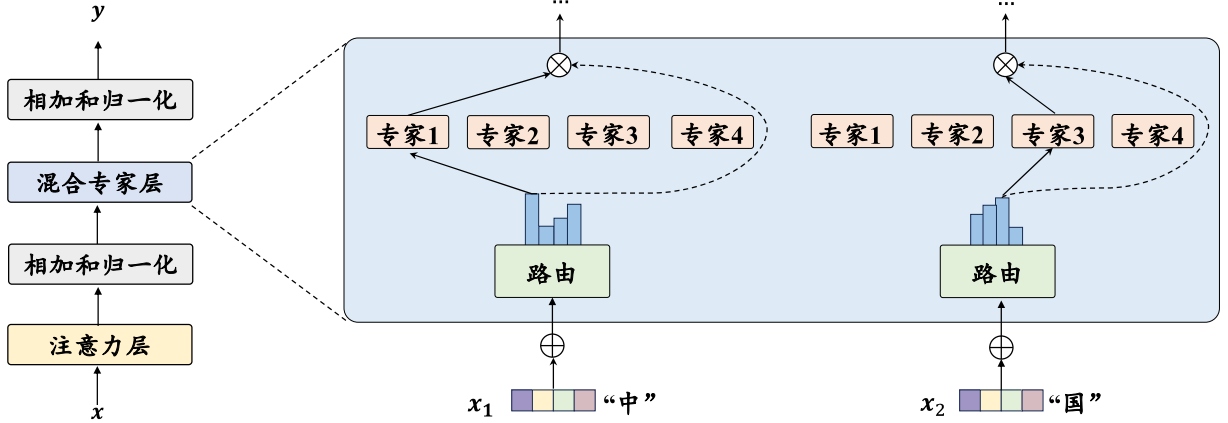


图 5.5 混合专家模型示意图

### 5.2.6 混合专家模型

如第 2.2 节所述，大语言模型能够通过扩展参数规模实现性能的提升。然而，随着模型参数规模的扩大，计算成本也随之增加。为了解决这一问题，研究人员在大语言模型中引入了基于稀疏激活的混合专家架构（Mixture-of-Experts, MoE），旨在不显著提升计算成本的同时实现对于模型参数的拓展。

在混合专家架构中，每个混合专家层包含  $K$  个专家组件，记为  $[E_1, E_2, \dots, E_K]$ ，其中每个专家组件  $E_i$  都是一个前馈神经网络。对于输入的每个词元表示  $\mathbf{x}_t$ ，模型通过一个路由网络（或称为门控函数） $G$  来计算该词元对应于各个专家的权重。在路由函数中，首先通过线性层  $\mathbf{W}^G \in \mathbb{R}^{H \times K}$  映射为  $K$  个专家的得分，并基于此选择出概率最高的  $k$  个专家进行激活。随后，这  $k$  个专家的得分将被送入 softmax 函数计算出它们的权重  $G(\mathbf{x}_t) = [G(\mathbf{x}_t)_1, \dots, G(\mathbf{x}_t)_k]$ ，没有被选择的专家权重将被置为 0。上述路由网络的计算过程如下式所示：

$$G(\mathbf{x}_t) = \text{softmax}(\text{topk}(\mathbf{x}_t \cdot \mathbf{W}^G)). \quad (5.35)$$

之后，每个被选择的词元的输出的加权和将作为该混合专家网络层的最终输出  $\mathbf{o}_t$ ：

$$\mathbf{o}_t = \text{MoELayer}(\mathbf{x}_t) = \sum_{i=1}^K G(\mathbf{x}_t)_i \cdot E_i(\mathbf{x}_t). \quad (5.36)$$

目前具有代表性的混合专家模型是 Mixtral (8×7B)，该模型在 Mistral (7B) 的基础上，使用了混合专家模块。具体来说，Mixtral 每一层都配备了 8 个专家 (7B)，并对每个词元选择 2 个专家进行后续计算。在每次计算被激活的参数仅仅有 13B 的情况下，其性能超越了更熟规模更大的 LLaMA-2 (70B)，进一步证明了混合专



家架构的有效性。下面给出了 Mixtral 混合专家层的一个 PyTorch 示例代码：

```

1 class MoeLayer(nn.Module):
2     def __init__(self, experts: List[nn.Module], gate: nn.Module,
3         num_experts_per_tok: int):
4         super().__init__()
5         assert len(experts) > 0
6         self.experts = nn.ModuleList(experts) # 所有专家列表
7         self.gate = gate # 路由网络
8         self.num_experts_per_tok = num_experts_per_tok # 每个词元选择的专家数
9         ↪ 目
10
11     def forward(self, inputs: torch.Tensor):
12         gate_logits = self.gate(inputs)
13         weights, selected_experts = torch.topk(gate_logits,
14             self.num_experts_per_tok)
15         # 使用路由网络选择出 top-k 个专家
16         weights = F.softmax(weights, dim=1,
17             ↪ dtype=torch.float).to(inputs.dtype)
18         # 计算出选择的专家的权重
19         results = torch.zeros_like(inputs)
20         for i, expert in enumerate(self.experts):
21             batch_idx, nth_expert = torch.where(selected_experts == i)
22             results[batch_idx] += weights[batch_idx, nth_expert, None] *
23             ↪ expert(
24                 inputs[batch_idx]
25             )
26         # 将每个专家的输出加权相加作为最终的输出
27         return results

```

### 5.2.7 LLaMA 的详细配置

综合本节讨论的内容，下面给出了关于模型详细配置的推荐建议。首先，为了增强模型的训练稳定性，建议采用前置的 RMSNorm 作为层归一化方法。其次，在选择激活函数时，为了获得更优的模型性能，可以优先考虑使用 SwiGLU 或 GeGLU。最后，对于位置编码，可以优先选择 RoPE 或者 ALiBi，这两种位置编码方法在建模长序列数据时通常能够具有较好的性能。接下来，我们以 LLaMA 模型的代码实现，来介绍 Transformer 解码器模型是如何进行模型搭建并且实现前向计算的过程。

对于一个 LLaMA 模型，其首先将输入的词元序列通过词嵌入矩阵转化为词向量序列。之后，词向量序列作为隐状态因此通过  $L$  个解码器层，并在最后使用 RMSNorm 进行归一化。归一化后的最后一层隐状态将作为输出。LLaMA 在 Transformers 库中的整体实现如下所示：

```

1 class LlamaModel(LlamaPreTrainedModel):
2     def __init__(self, config: LlamaConfig):
3         super().__init__(config)
4         self.vocab_size = config.vocab_size
5         # LLaMA 的词表大小
6         self.embed_tokens = nn.Embedding(config.vocab_size,
7             ↪ config.hidden_size, self.padding_idx)
8         # LLaMA 的词嵌入矩阵, 将输入的 id 序列转化为词向量序列
9         self.layers = nn.ModuleList(
10             [LlamaDecoderLayer(config, layer_idx) for layer_idx in
11              ↪ range(config.num_hidden_layers)]
12         )
13         # 所有的 Transformer 解码器层
14         self.norm = LlamaRMSNorm(config.hidden_size,
15             ↪ eps=config.rms_norm_eps)
16         causal_mask = torch.full(
17             (config.max_position_embeddings,
18              ↪ config.max_position_embeddings), fill_value=True,
19             ↪ dtype=torch.bool
20         )
21
22 @add_start_docstrings_to_model_forward(Llama_INPUTS_DOCSTRING)
23 def forward(
24     self,
25     input_ids: torch.LongTensor = None,
26     attention_mask: Optional[torch.Tensor] = None,
27     position_ids: Optional[torch.LongTensor] = None,
28     **kwargs,
29 ) -> Union[Tuple, BaseModelOutputWithPast]:
30     if inputs_embeds is None:
31         inputs_embeds = self.embed_tokens(input_ids)
32         # 将输入的 input id 序列转化为词向量序列
33     causal_mask = self._update_causal_mask(attention_mask,
34         ↪ inputs_embeds)
35     # 创建单向注意力的注意力掩盖矩阵
36
37     hidden_states = inputs_embeds
38
39     for decoder_layer in self.layers:
40         hidden_states = decoder_layer(
41             hidden_states,
42             attention_mask=causal_mask,
43             position_ids=position_ids,
44         )[0]
45         # 用每个 LLaMA 解码器层对词元的隐状态进行映射
46     hidden_states = self.norm(hidden_states)
47     # 对每个词元的隐状态使用 RMSNorm 归一化
48     return BaseModelOutputWithPast(
49         last_hidden_state=hidden_states,
50     )

```

在每个解码器层中，隐状态首先通过层前的 RMSNorm 归一化并被送入注意力模块。注意力模块的输出将和归一化前的隐状态做残差连接。之后，新的隐状态进行 RMSNorm 归一化，并送入前馈网络层。和上面一样，前馈网络层的输出

同样做残差连接，并作为解码器层的输出。Transformers 库中 LLaMA 每一层的代码实现如下所示：

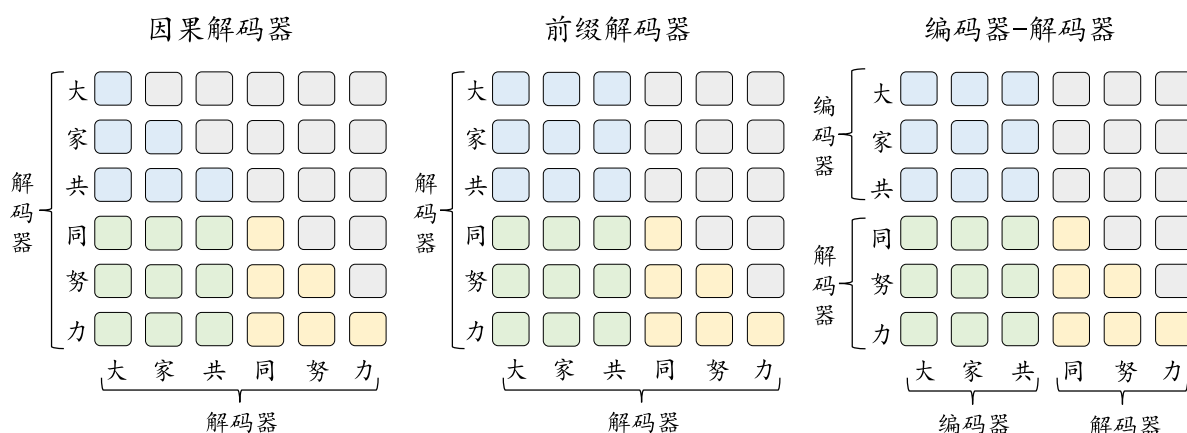
```

1 class LlamaDecoderLayer(nn.Module):
2     def __init__(self, config: LlamaConfig, layer_idx: int):
3         super().__init__()
4
5         self.hidden_size = config.hidden_size
6         self.self_attn = LlamaAttention(config=config, layer_idx=layer_idx)
7         ↪ # 注意力层
8         self.mlp = LlamaMLP(config) # 前馈网络层
9
10        self.input_layernorm = LlamaRMSNorm(config.hidden_size,
11        ↪ eps=config.rms_norm_eps)
12        self.post_attention_layernorm = LlamaRMSNorm(config.hidden_size,
13        ↪ eps=config.rms_norm_eps)
14        # 注意力层和前馈网络层前的 RMSNorm
15
16    def forward(
17        self,
18        hidden_states: torch.Tensor,
19        attention_mask: Optional[torch.Tensor] = None,
20        position_ids: Optional[torch.LongTensor] = None,
21        **kwargs,
22    ) -> Tuple[torch.FloatTensor, Optional[Tuple[torch.FloatTensor,
23    ↪ torch.FloatTensor]]]:
24
25        residual = hidden_states
26
27        hidden_states = self.input_layernorm(hidden_states)
28        # 注意力层前使用 RMSNorm 进行归一化
29        hidden_states, self_attn_weights, present_key_value =
30        ↪ self.self_attn(
31            hidden_states=hidden_states,
32            attention_mask=attention_mask,
33            position_ids=position_ids,
34            **kwargs,
35        )
36        # 进行注意力模块的计算
37        hidden_states = residual + hidden_states
38        # 残差连接
39
40        residual = hidden_states
41        hidden_states = self.post_attention_layernorm(hidden_states)
42        # 前馈网络层前使用 RMSNorm 进行归一化
43        hidden_states = self.mlp(hidden_states)
44        # 进行前馈网络层的计算
45        hidden_states = residual + hidden_states
46        # 残差连接
47        outputs = (hidden_states,)
48        return outputs

```

## 5.3 主流架构

在预训练语言模型时代，自然语言处理领域广泛采用了预训练 + 微调的范式，并诞生了以 BERT 为代表的编码器（Encoder-only）架构、以 GPT 为代表的解码器（Decoder-only）架构和以 T5 为代表的编码器-解码器（Encoder-decoder）架构的大规模预训练语言模型。随着 GPT 系列模型的成功发展，当前自然语言处理领域走向了生成式大语言模型的道路，解码器架构已经成为了目前大语言模型的主流架构。进一步，解码器架构还可以细分为三个变种架构，包括因果解码器（Causal Decoder）架构和前缀解码器（Prefix Decoder）架构。值得注意的是，学术界所提到解码器架构时，通常指的都是因果解码器架构。图 5.6 针对这三种架构进行了对比。



**图 5.6** 三种主流架构的注意力模式比较示意图（蓝色、绿色、黄色和灰色的圆角矩形分别表示前缀词元之间的注意力、前缀词元和目标词元之间的注意力、目标词元之间的注意力以及掩码注意力, 图片来源: [10])

### 5.3.1 编码器-解码器架构

编码器-解码器架构是自然语言处理领域里一种经典的模型结构，广泛应用于如机器翻译等多项任务。原始的 Transformer 模型也使用了这一架构，组合了两个分别担任编码器和解码器的 Transformer 模块（详细阐述参见第 5.1 节）。如图 5.6 所示，此架构在编码器端采用了双向自注意力机制对输入信息进行编码处理，而在解码器端则使用了交叉注意力与掩码自注意力机制，进而通过自回归的方式对输出进行生成。基于编码器-解码器设计的预训练语言模型（诸如 T5 [77] 等）在众多自然语言理解与生成任务中展现出了优异的性能，但是目前只有如 FLAN-T5 [39]

等少数大语言模型是基于编码器-解码器架构构建而成的。

### 5.3.2 因果解码器架构

当前，绝大部分主流的大语言模型采用了因果解码器架构。因果解码器采用了 Transformer 中的解码器组件，同时做出了几点重要改动。首先，因果解码器没有显式地区分输入和输出部分。如图 5.6 所示，该架构采用了单向的掩码注意力机制，使得每个输入的 token 只关注序列中位于它前面的 token 和它本身，进而自回归地预测输出的 token。此外，由于不含有编码器部分，因果解码器删除了关注编码器表示的交叉注意力模块。经过自注意力模块后的 token 表示将直接送入到前馈神经网络中。在因果解码器架构中，最具有代表性的模型就是 OpenAI 推出的 GPT 系列。其中，GPT-3 将模型参数拓展到了 100B 级别，并展现出了强大的零样本和少样本学习能力。伴随着 GPT-3 的成功，因果解码器被广泛采用于各种大语言模型中，包括 BLOOM、LLaMA 和 Mistral 等。

### 5.3.3 前缀解码器架构

前缀解码器架构也被称为非因果解码器架构，对于因果解码器的掩码机制进行了修改。该架构和因果解码器一样，仅仅使用了解码器组件。与之不同的是，该架构参考了编码器-解码器的设计，对于输入和输出部分进行了特定处理。如图 5.6 所示，前缀解码器对于输入（前缀）部分使用双向注意力进行编码，而对于输出部分利用单向的掩码注意力利用该 token 本身和前面的 token 进行自回归地预测。与编码器-解码器不同的是，前缀解码器在编码和解码过程中是共享参数的，并没有划分为独立的解码器和编码器。对于前缀解码器，也可以由现有的因果解码器继续预训练转换而来，进而加速该模型的训练。例如，U-PaLM [175] 是从 PaLM [33] 继续预训练而来的。当前，基于前缀解码器架构的代表性大语言模型包括 GLM-130B [162] 和 U-PaLM [175]。

## 5.4 长上下文模型

在实际应用中，大语言模型对于长文本数据的处理需求日益凸显，尤其在长文档分析、多轮对话、故事创作等场景下。在这些情况下，模型需要处理的文本的长度常常超出预定义上下文窗口大小。例如，LLaMA-2 的上下文窗口限制为 4,096

个词元。为了支持长文本处理，多家机构均已推出面向具有超长上下文窗口的大语言模型或 API。例如，OpenAI 发布了支持 128K 上下文窗口的 GPT-4 Turbo，而 Anthropic 则推出了具有 200K 上下文窗口的 Claude-2.1。

给定一个预训练后的大语言模型，如何有效拓展其上下文窗口以应对更长的文本数据成为当前学术界的研究焦点。目前，增强大语言模型长文本建模能力的研究主要集中在两个方向：一是扩展位置编码（详见第 5.4.1 节），二是调整上下文窗口（详见第 5.4.2 节）。除了探讨拓展上下文窗口的方法外，本部分将在最后探讨训练长上下文模型所需的长文本数据（详见第 5.4.3 节）。

### 5.4.1 扩展位置编码

在基于 Transformer 架构的大语言模型中，模型的上下文建模能力通常受到训练集中文本数据长度分布的限制。一旦超出这个分布范围，模型的位置编码往往无法得到充分训练，从而导致模型处理长文本的性能下降。因此，当大语言模型面临超出其最大训练长度的任务时，需要对于位置编码进行扩展，以适应更长的绝对或相对位置。

实际上，某些特定的位置编码在超出原始上下文窗口的文本上，也能够表现出较好的建模能力，这种能力通常被称为外推（Extrapolation）能力。在已有的基于相对位置的位置编码方法中，T5 偏置 [77]、ALiBi [170] 以及 xPos [176] 等方法都展现出了不同程度的外推能力。值得注意的是，尽管这种外推能力可以确保模型在长文本上继续生成流畅的文本，但模型对长文本本身的理解能力可能无法达到与短文本相同的水平。为了真正增强长文本建模能力，通常还需要在更长的文本上进行一定的训练。

然而，目前比较主流的位置编码方法 RoPE 在未经特殊修改的情况下并不具备良好的外推能力。具体来说，在处理更长的文本时，RoPE 在每个子空间上需要处理更大的旋转角度，而这些旋转角度可能会超过其训练中的角度分布范围。因此，很多研究工作在 RoPE 的基础上进行了重要改进，旨在提升其在不经过训练或继续训练的情况下对于长文本的建模能力。接下来将为这些改进方法给出一个统一的形式化定义（关于 RoPE 的详细介绍，请参阅第 5.2.4 节）。

形式化来说，对于一个原始上下文窗口为  $T_{\max}$  的模型，目标是将其上下文窗口扩展到  $T'_{\max}$ （其中  $T'_{\max} > T_{\max}$ ）。在 RoPE 的每个子空间  $i$  上，对于相对位置  $t$ ，旋转角度  $f(t, i) = t \cdot \theta_i$  的修改可以分解为对距离  $t$  的修改  $g(t)$  和对基  $\theta_i$  的修改



$h(i)$ 。因此，新的旋转角度可以表示为如下形式：

$$f(t, i) = g(t) \cdot h(i). \quad (5.37)$$

### 直接微调

为了使大语言模型适应更长的上下文长度，一种直接的策略是使用相应的长文本数据对于模型进行微调。在这种情况下，模型可以直接根据相对位置计算出对应的位置编码，而无需对 RoPE 本身进行任何修改。旋转角度的计算方式依旧和之前相同：

$$f(t, i) = t \cdot \theta_i. \quad (5.38)$$

然而，在更长的文本上进行训练会导致出现比原始上下文窗口内更大的最大旋转角度  $T'_{\max} \cdot \theta_i$ 。在模型进行微调前，这些超出原始窗口的位置对应的注意力值会远大于窗口内的值。因此，如果不修改 RoPE 而直接在长文本数据上进行微调，通常会导致收敛缓慢，并需要大量数据进行继续预训练。

### 位置索引修改

鉴于直接微调可能引发旋转角度增大和注意力值爆炸的问题，有必要对旋转角度施加限制，以确保拓展后的上下文窗口中的旋转角度得到充分且有效的训练。为实现这一目标，可以通过修改位置索引  $g(t)$  来调整所有子空间的旋转角度，从而保证其不超过原始上下文窗口所允许的最大值。具体来说，位置索引的修改可采用以下两种方法：

- **位置内插**. 位置内插 [157] 方法对于位置索引进行特定比例的缩放，以保证旋转角度不会超过原始上下文窗口的最大值。具体来说，该策略将所有位置索引乘以一个小于 1 的系数  $T_{\max}/T'_{\max}$ （其中  $T_{\max} < T'_{\max}$ ）， $T_{\max}$  和  $T'_{\max}$  分别表示原始上下文窗口和拓展后的上下文窗口的长度。通过进行这样的缩放，新的旋转角度计算公式变为：

$$g(t) = \frac{T_{\max}}{T'_{\max}} \cdot t. \quad (5.39)$$

通常来说，使用位置内插方法进行微调的训练代价较小。例如，只需要一千步左右的训练就可以将 LLaMA (33B) 的模型的上下文窗口长度从 2,048 拓展到 8,192 个词元 [157]。然而在处理较短的文本时，由于位置索引的缩放，可能会对模型的性能产生一定的负面影响。

- **位置截断**. 不同于位置内插，位置截断针对不同距离采用了不同的处理方

式。该方法依据语言建模的局部性原理，对模型中近距离敏感的位置索引进行保留，同时截断或插值处理远距离的位置索引，确保其不超出预设的最大旋转角度。具体来说，采用位置截断的 ReRoPE 和 LeakyReRoPE [177] 方法首先设定一个不大于原始上下文窗口长度的窗口大小  $w$  ( $w \leq T_{\max}$ )。在此窗口范围内的部分，仍使用原始相对位置索引；对于超出此窗口的部分，位置索引则会被截断至窗口大小，即  $g(t) = w$ ；或通过线性插值方式，将目标上下文窗口长度的位置索引映射回原始上下文窗口长度，即  $g(t) = w + \frac{T_{\max}-w}{T'_{\max}-w} \cdot (t - w)$ 。上述位置截断方法可通过以下公式表达：

$$g(t) = \begin{cases} t & t \leq w, \\ w & t > w \text{ 且使用 ReRoPE}, \\ w + \frac{(T_{\max}-w)(t-w)}{T'_{\max}-w} & t > w \text{ 且使用 LeakyReRoPE}. \end{cases} \quad (5.40)$$

通过这种方法对 RoPE 进行修改后，模型能够直接应用于更长的上下文而无需重新训练，并且依然保持对短文本的建模能力。然而，这种方法需要对注意力矩阵进行二次计算，进而增加了额外的计算开销。

### 基修改

根据第 5.2.4 节中对 RoPE 的介绍，每个子空间  $i$  都有一个对应的波长  $\lambda_i$ ，表示在该子空间上旋转一周所需要的距离。然而，某些子空间的波长可能会超过上下文窗口的长度 ( $\lambda_i > T_{\max}$ )，导致模型在这些子空间上无法对完整的旋转周期进行训练。这些子空间通常被称为关键子空间 [178]。在面临更长的文本时，RoPE 关键子空间的旋转角度对应的正余弦函数值并没有在训练阶段出现过，这就容易导致注意力值出现异常。因此，如果想要调整这些子空间的旋转角度分布，另一种方法是针对这些子空间的基  $h(i)$  进行缩放：

$$f(T'_{\max}, i) = T'_{\max} \cdot h(i) \leq T_{\max} \cdot \theta_i. \quad (5.41)$$

对基的修改可以通过对基的底数修改以及对基的截断实现，下面介绍这些修改方法。

- 底数调整. 依据公式  $\theta_i = b^{-2(i-1)/H}$  (参见方程 (5.32))，通过调整底数可以改变旋转的角度。具体来说，按照一定比例增大底数可以对基进行缩小，从而缩小旋转的角度，使得模型在不经额外训练的情况下能够处理更长的上下文窗口 [179]。在这种情况下，每个子空间的旋转角度由下式给出：

$$h(i) = (\alpha \cdot b)^{-(i-1)/H}, \quad (5.42)$$

其中,  $\alpha$  是一个大于等于放缩比例的数, 通过对底数进行增大, 实现缩小基来处理更长文本的能力。在实践中, 不同方法通常会采用不同的  $\alpha$  值。例如, NTK-RoPE 基于目标上下文窗口, 将  $\alpha$  设置为  $(T'_{\max}/T_{\max})^{H/H-2}$ ; 而 Dynamic-NTK-RoPE 则根据输入文本长度  $T$  动态地将窗口大小进行调整  $\alpha = \max(1, T/T_{\max})$ 。如果要进一步提升模型的长文本建模能力, 还可以在长文本数据上进行微调。此时, 使用较大的底数 (例如,  $b = 10^8$ ) 通常能够获得更好的性能。

- 基截断. 与底数调整相似, 基截断方法通过修改关键子空间来避免产生过大的旋转角度 [180]。这种方法首先设定两个阈值  $a$  和  $c$ 。根据每个子空间上的基  $\theta_i$  与这两个阈值的比较结果, 可以选择相应的调整策略来对基进行调整: 当  $\theta_i \geq c$  时, 基的值会被保持不变; 当  $\theta_i \leq a$  时, 基会被设置为零; 当  $a < \theta_i < c$  时, 基会被截断为一个较小的固定数值。通过上述的基截断操作, 可以有效地防止在位置索引较大时出现超出预期分布的旋转角度, 从而有助于实现更好的模型外推性能。然而, 这种方法在一定程度上削弱了某些子空间对不同位置索引的区分能力, 进而可能对模型的性能产生不利影响。该方法的数学表达式如下:

$$h(i) = \begin{cases} \theta_i & \theta_i \geq c \\ \beta & c \geq \theta_i \geq a \\ 0 & \theta_i \leq a. \end{cases} \quad (5.43)$$

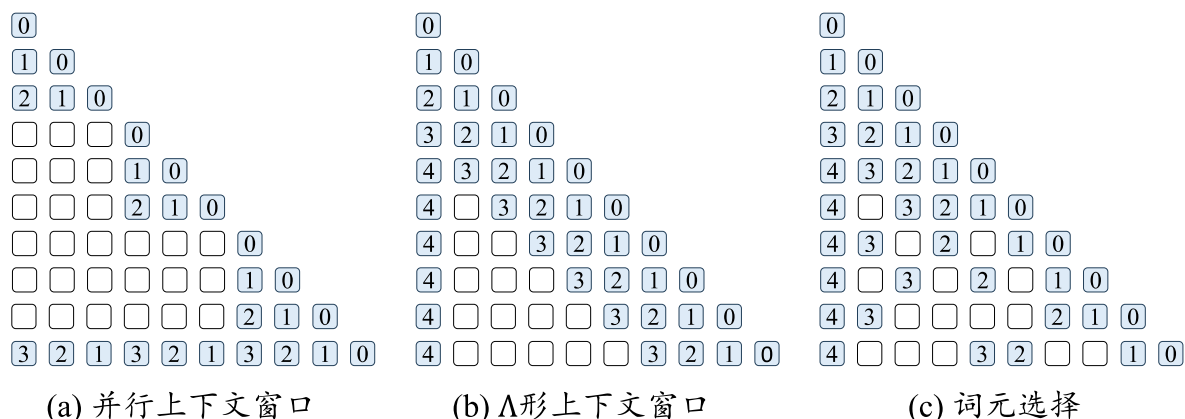
### 5.4.2 调整上下文窗口

为了解决 Transformer 架构对于上下文窗口的限制, 除了使用扩展位置编码来拓宽上下文窗口外, 另一种行之有效的策略是采用受限的注意力机制来调整原始的上下文窗口, 从而实现对更长文本的有效建模。下面将详细介绍三种调整上下文窗口的方法。

#### 并行上下文窗口

并行上下文窗口方法 [181] 采用了一种分而治之的策略来处理输入文本。具体来说, 该方法将输入文本划分为若干个片段, 每个片段都进行独立的编码处理, 并共享相同的位置编码信息。在生成阶段, 通过调整注意力掩码, 使得后续生成的词元能够访问到前序的所有词元。然而, 该方法无法有效地区分不同段落之间的顺序关系, 在某些特定任务上可能会限制模型的表现能力。

#### $\Delta$ 形上下文窗口



**图 5.7** 三种调整上下文窗口方法的示意图（白色表示被掩盖的词元，蓝色表示进行注意力计算的词元，块上面的数字表示位置编码的相对位置）

在处理长文本时，大语言模型有时会表现出一种不均匀关注的现象：它们倾向于对序列起始位置以及邻近的词元赋予更高的注意力权重。基于这一观察，StreamingLLM [182] 等工作引入了“ $\Lambda$  形”注意力掩码方法，能够有选择性地关注每个查询的邻近词元以及序列起始的词元，同时忽略超出这一范围的其他词元。在给定的有限内存资源下，这种方法能够生成几乎无限长的流畅文本。然而，由于无法有效利用被忽略的词元信息，这种方法无法充分利用所有的上下文信息。

### 词元选择

在 Transformer 的注意力模块中，对于每个词元的预测，并非所有先前词元都提供等量的贡献。实际上，小部分紧密相关词元的注意力分数总和就能够接近所有词元的注意力分数总和。基于这样的一个实践观察，相关研究工作提出了词元选择方法，旨在挑选出最重要的  $k$  个词元，以实现对于完整注意力的有效拟合。词元选择方法可以通过查询与词元相似度和查询与词元所在分块的相似度实现。

- 查询与词元相似度. 在此类方法中，根据位置索引和上下文窗口，词元被划分为窗口内的近距离词元和窗口外的远距离词元。对于窗口外的远距离词元，通常利用外部存储保存它们的键值对，并采用  $k$  近邻搜索方法来获取当前生成所需的  $T_{\max}$  个最相关词元 [152]。具体来说，在 Transformer 模型中，可以首先选定若干层，针对这些层从外部存储中检索到最相关词元的键值对，进一步将其送入注意力计算模块中，为模型补充远程语义信息；而在其他层中，模型仍然针对上下文窗口内的词元进行注意力计算。

- 查询与分块相似度. 分块级别的词元选择将序列划分为不同的长度固定的分

块，并从分块序列中选择出最相关的部分分块 [183]。具体来说，模型首先将每个分块中所有的隐状态压缩为一个键向量表示，然后利用  $k$  近邻方法选出与查询最相关的  $k$  个分块，并保证这些块中的总词元数目至多为  $T_{\max}$ 。这些分块中所有的词元将按照它们在整个输入序列中的出现的顺序进行排序，并按照排序后的位置赋予位置编码。随后，这些词元被送入注意力模块中处理。与词元级别的方法不同，分块级别的选择通常不需要外部存储，而是将所有数据存储在内存中。此外，不同的层和头可以根据自身的特性选择不同的词元集合，从而更为灵活地利用整个长序列的信息。因此，分块级别的词元选择能够在保证性能的同时降低计算复杂度和内存需求。

### 5.4.3 长文本数据

为了有效拓展模型的长文本建模能力，通常需要使用特殊准备的数据对于模型进行继续预训练。本节将详细介绍如何确定所需的长文本数据量，以及如何合理分布长文本数据的领域，以确保模型的长文本建模能力。

- 长文本数据量. 标准的预训练任务通常需要使用大量的文本数据。而对于面向长文本建模的继续预训练来说，可以采用少量长文本数据进行轻量化的继续预训练。这一方法需要模型在初始预训练阶段已经学会了利用远程词元信息的能力，仅需使模型适应更长的上下文窗口。一般而来说，只需在约 1B 级别的词元上执行数百步的训练，就可以将 7B 或者 13B 大小的 LLaMA 系列模型的上下文窗口至 100K 词元以上的长度，并具有较好的长上下文利用能力 [184, 185]。然而，值得注意的是，模型在处理短文本时的性能可能会受到一定程度的影响。

- 长文本数据混合. 除了数据总量外，训练数据集中不同数据的混合也是影响模型性能的关键因素，主要包括长文本的领域分布和长文本的类型。在预训练数据中，不同领域长文本的比例存在差异。一般而言，书籍、科学论文、代码仓库等领域包含较多的长文本数据。直接对这些长文本数据采样进行进一步继续预训练可能会导致与预训练数据分布的不匹配，导致模型过多的学习了某一领域长文本的特征，从而损害了在其他领域的影响。为了提升模型的泛化能力，长文本数据的领域应尽可能多样化，并且与预训练数据集的分布保持相似 [185]。除了数据的领域分布外，数据本身的语义特性也是数据混合需要考虑的问题。在 LongWanjuan [186] 中，研究人员基于连贯性、衔接性和复杂性将长文本数据分为整体型（完整的有意义的长文）、聚合型（多篇相关文本的聚合）和杂乱型（杂乱无章的文本）。实



验结果显示，通过去除杂乱型的文本，并在保留整体型文本的同时对聚合型文本进行上采样构建的训练集，可以更好地提升模型的长文本建模能力。

## 5.5 新型模型架构

Transformer 模型自问世以来，在自然语言处理、计算机视觉等多个领域得到了广泛应用，并展现出卓越的数据表示与建模能力。然而，Transformer 的自注意力机制在计算每个词元时都需要利用到序列中所有词元的信息，这导致计算和存储复杂度随输入序列长度的平方级别增长。在处理长序列时，这种复杂性会消耗大量的计算资源与存储空间。为了解决这个问题，研究人员致力于新型模型架构的设计。这些新型模型大多基于参数化状态空间模型 (State Space Model, SSM) 进行设计，在长文本建模效率方面相比 Transformer 有了大幅改进，同时也保持了较好的序列建模能力。在本节中，我们将首先对于参数化状态空间模型展开讨论，然后针对状态空间模型的各种变种模型进行介绍。为了帮助读者更好地理解这些模型之间的区别，我们在表 5.2 中对于它们进行了比较。

**表 5.2** 不同模型的比较 (T 表示序列长度, H 表示输入表示的维度, N 表示状态空间模型压缩后的维度, M 表示 Hyena 每个模块的层数)

| 模型          | 可并行性 | 解码复杂度            | 训练复杂度                         |
|-------------|------|------------------|-------------------------------|
| Transformer | ✓    | $O(TH + H^2)$    | $O(T^2H + TH^2)$              |
| 标准 SSM      | ✓    | $O(N^2H + H^2)$  | $O(TH \log T + THN^2 + TH^2)$ |
| Mamba       | ×    | $O(N^2H + H^2)$  | $O(TN^2H + TH^2)$             |
| RWKV        | ×    | $O(H^2)$         | $O(TH^2)$                     |
| RetNet      | ✓    | $O(H^2)$         | $O(TH^2)$                     |
| Hyena       | ✓    | $O(TM H + MH^2)$ | $O(TM H \log T + TM H^2)$     |

### 5.5.1 参数化状态空间模型

状态空间模型是一种动态时域模型，在控制系统、经济学等多个领域都有着广泛应用。近年来，深度学习领域也开始引入参数化状态空间模型对于序列数据进行建模。通俗来说，参数化状态空间模型可以看作是循环神经网络和卷积神经网络的“结合体”。一方面，该模型可以利用卷积计算对输入进行并行化编码。另一方面，该模型在计算中不需要访问前序的所有词元，仅仅利用前一个词元就可以自回归地进行预测。因此，该模型在解码时展现出了更高的计算效率。由于自



然语言文本本质上是离散型序列数据，本书将着重探讨离散型状态空间模型。

为了同时实现并行化计算和循环解码，状态空间模型在输入和输出之间引入了额外的状态变量。在循环计算中，状态空间模型首先循环地利用当前时刻的输入  $\mathbf{x}_t$  和前一个时刻的状态  $\mathbf{S}_{t-1}$  对当前时刻的状态  $\mathbf{S}_t$  进行计算。然后，该模型将当前时刻的状态  $\mathbf{S}_t$  进一步映射为输出  $\mathbf{y}_t$ 。该模型的数学表示如下所示：

$$\begin{aligned}\mathbf{S}_t &= \mathbf{A} \otimes \mathbf{S}_{t-1} + \mathbf{B} \otimes \mathbf{x}_t, \\ \mathbf{y}_t &= \mathbf{C} \otimes \mathbf{S}_t,\end{aligned}\tag{5.44}$$

其中， $\mathbf{A} \in \mathbb{R}^{H \times N \times N}$ 、 $\mathbf{B} \in \mathbb{R}^{H \times N \times 1}$  和  $\mathbf{C} \in \mathbb{R}^{H \times 1 \times N}$  是可学习参数，而  $\otimes$  表示批量矩阵乘法。针对上述公式，当前时刻的输出可以通过循环的方式进行分解，进而表示为如下的数学形式：

$$\begin{aligned}\mathbf{y}_t &= \mathbf{C} \otimes \mathbf{S}_t = \mathbf{C} \otimes \mathbf{A} \otimes (\mathbf{A} \otimes \mathbf{S}_{t-2} + \mathbf{B} \otimes \mathbf{x}_{t-1}) + \mathbf{C} \otimes \mathbf{B} \otimes \mathbf{x}_t \\ &= \mathbf{C} \otimes \mathbf{A}^{t-1} \otimes \mathbf{B} \mathbf{x}_1 + \cdots + \mathbf{C} \otimes \mathbf{B} \otimes \mathbf{x}_t = \sum_{i=1}^t \mathbf{C} \otimes \mathbf{A}^{t-i} \otimes \mathbf{B} \mathbf{x}_i.\end{aligned}\tag{5.45}$$

根据卷积计算的定义，公式 5.45 对输出  $\mathbf{y}_t$  的计算可以看作是对输入的卷积，其中卷积核为  $\mathbf{K}$ 。这一计算可以表示为：

$$\begin{aligned}\mathbf{K} &= (\mathbf{C} \otimes \mathbf{B}, \mathbf{C} \otimes \mathbf{A} \otimes \mathbf{B}, \dots, \mathbf{C} \otimes \mathbf{A}^{t-1} \otimes \mathbf{B}, \dots), \\ \mathbf{y} &= \mathbf{x} * \mathbf{K},\end{aligned}\tag{5.46}$$

其中，“\*”表示卷积计算。在使用卷积计算时，状态空间模型可以利用快速傅里叶变换加速计算效率，从而通过  $O(TH \log T + THN^2 + TH^2)$  的复杂度建模整个序列。在循环计算的时候，状态空间模型不需要和 Transformer 一样对前面所有时刻的状态进行访问，而是仅仅需要前一个时刻的状态。因此，该模型仅仅需要  $O(N^2H + H^2)$  的复杂度就可以完成对整个序列的建模。由于具有更优的计算效率，状态空间模型常常被用来对长序列数据进行建模。

### 5.5.2 状态空间模型变种

尽管状态空间模型计算效率较高，但是在文本任务上的表现相比 Transformer 模型仍有一定的差距。为此，一系列研究工作对于状态空间模型进行了性能改进，在保证计算效率的同时提高其语言建模的能力。代表性模型包括 Mamba [187]、RWKV (Receptance Weighted Key Value) [188]、RetNet (Retentive Network) [189] 和 Hyena [190] 等。接下来，我们将对这些模型进行简要介绍。

• *Mamba*. Mamba [187] 是一种状态空间模型的变种，主要思想是在状态空间模型的状态更新（公式 5.44）中引入了基于当前输入的信息选择（Selection）机制，来确定当前时刻状态如何从前一时刻状态以及当前输入中提取信息，从而提升其在语言建模上的性能。标准的状态空间模型在每次更新状态  $S_t$  的时候，都对输入  $x_t$  和前一个时刻的状态  $S_{t-1}$  使用相同的线性映射参数（公式 5.44）。然而，对于文本建模而言，模型需要能够自适应地基于输入和之前状态来实现更好的上下文表示效果。因此，Mamba 提出将更新状态和输出的方程中（公式 5.44）的参数矩阵  $(A, B, C)$  表示成输入  $x_t$  的非线性函数。进而，模型能够基于当前时刻的输入  $x_t$  对上一时刻的状态  $S_{t-1}$  和当前时刻输入  $x_t$  中的信息进行选择性过滤，从而实现更为有效的上下文表示。相比于标准状态空间模型，Mamba 展现出了更好的文本建模性能，但是由于引入了关于  $x_t$  的非线性关系，Mamba 无法利用快速傅里叶变换实现高效卷积计算。

• *RWKV*. RWKV [188] 尝试将 RNN 和 Transformer 的优点进行结合，继承了 Transformer 的建模优势和 RNN 的计算效率。作为一个主要技术创新，RWKV 在每层的计算中使用词元偏移（Token Shift）来代替词元表示。在每一步的状态计算中，它显示地引入了上一个词元  $x_{t-1}$ ，通过两个相邻的词元  $x_t$  和  $x_{t-1}$  进行线性插值来代替  $x_t$  作为后续模块的输入。进一步，RWKV 将 Transformer 中的多头注意力模块和前馈网络模块分别替换为时间混合（Time-mixing）模块和频道混合（Channel-mixing）模块。其中，时间混合模块是一个类似于门控的 RNN 的网络，并使用词元偏移对状态进行更新；频道混合模块是在前馈网络的基础上引入了词元偏移进行映射。类似于 Mamba，RWKV 在解码过程中可以像 RNN 一样只参考前一时刻的状态，但是在训练过程中缺乏并行计算的能力。

• *RetNet*. RetNet [189] 提出使用多尺度保留（Multi-scale Retention, MSR）机制来代替多头注意力模块，从而提升计算效率。多尺度保留机制是在标准状态空间模型的基础上，在状态更新的线性映射中引入了输入相关信息来提升序列建模能力。每个保留模块中，输入词元被映射为查询向量  $q_t$ 、键向量  $k_t$  和值向量  $v_t$ ，并通过  $k_t^T v_t$  和前一个时刻的状态  $S_{t-1}$  进行线性相加，得到当前的状态： $S_t = AS_{t-1} + k_t^T v_t$ 。最后，RetNet 使用查询  $q_t$  将当前状态  $S_t$  映射为输出  $o_t = q_t S_t$ 。此外，RetNet 还可以通过类似注意力操作的矩阵乘法，对所有词元的状态进行并行化计算。因此类似于标准状态空间模型，RetNet 同时保留了循环计算和并行计算的优点。

• *Hyena*. Hyena [190] 提出使用长卷积模块（Long Convolution）来替换 Trans-

former 架构中的注意力模块，从而借助卷积的快速傅里叶变换来提高计算效率。Hyena 在每层的长卷积模块中包含了  $M$  个滤波器，即每个相对位置  $t$  有一个相应的滤波器  $\mathbf{h}(t)$ ，然后将这些滤波器组合成卷积核  $\mathbf{K} = (\mathbf{h}(1), \dots, \mathbf{h}(T))$ 。利用该卷积核与输入序列  $[\mathbf{x}_1, \dots, \mathbf{x}_t]$  进行卷积，可以对序列中不同位置的信息进行聚合，得到每个位置的中间表示  $\mathbf{z}_t$ 。最后，再使用门控函数  $\mathbf{g}(t)$ （基于输入  $\mathbf{x}_t$ ）对中间表示  $\mathbf{z}_t$  进行加权，得到该模块的最终输出。由于使用了卷积计算可以使用快速傅里叶变换进行加速，在训练中，Hyena 可以实现  $O(TM H \log T + TM H^2)$  的计算复杂度。但是在解码时，每次计算必须对前面所有的词元进行卷积，因此解码复杂度为  $O(TM H + M H^2)$ 。